

Sprint 05 Structured Notes: `useState`, Rendering, Immutable State Updates, and Beginner `useReducer`

0. Where You Are Before Sprint 05

Before Sprint 05, you should already understand these ideas:

- React builds user interfaces using components.
- JSX lets you write UI inside JavaScript.
- Props pass data from parent to child.
- `children` lets a component wrap other JSX.
- `map()` renders lists.
- Stable keys help React track list items.
- Events let React respond to clicks, typing, and form submissions.
- Controlled inputs store form values in React state.
- `event.preventDefault()` stops form reloads.

Sprint 05 is the point where React becomes properly **state-driven**.

Before this sprint, your app can show data and respond to user actions. After this sprint, your app can remember changing data, update the screen correctly, and avoid common mutation bugs.

1. Sprint 05 Goal

The goal of Sprint 05 is to understand how React remembers data and updates the UI.

By the end of this sprint, you should be able to:

- use `useState` for dynamic UI,
- explain what state is,
- explain React's render cycle,
- explain trigger, render, and commit,
- understand that each component instance has private state,
- call hooks at the top level only,
- update state based on previous state,
- update object state without losing fields,
- update array state without mutation,
- add items with spread,
- remove items with `filter()`,
- update items with `map()`,

- explain why direct mutation causes bugs,
- lift state up when sibling components need shared state,
- understand beginner `useReducer`,
- know when `useReducer` is clearer than `useState`,
- build a Packing List app.

Simple version:

Sprint 05 teaches you how React remembers changing data and updates the screen safely.

2. Big Mental Model

React apps often work like this:

```
User does something
↓
State changes
↓
React re-renders the component
↓
The screen updates
```

Example:

```
User clicks a button
↓
setCount runs
↓
count changes
↓
React renders again
↓
The new count appears
```

In Sprint 04, you used code like this:

```
<input value={name} onChange={(event) => setName(event.target.value)} />
```

That already used state. Sprint 05 explains state properly.

3. What Is State?

State is data that React remembers inside a component.

State can change over time.

When state changes, React updates the UI.

Simple definition:

State is React's memory for data that can change.

Examples of state:

```
counter number  
form input value  
cart items  
logged-in user  
theme mode  
selected tab  
todo list  
error message  
loading status  
packing list items
```

Props vs State

```
Props = data a component receives  
State = data a component owns and changes
```

Example of props:

```
function UserCard({ name }) {  
  return <h2>{name}</h2>;  
}
```

Example of state:

```
const [count, setCount] = useState(0);
```

4. Basic useState

useState is a React Hook.

It lets a component remember changing data.

Import it like this:

```
import { useState } from "react";
```

Basic pattern:

```
const [value, setValue] = useState(initialValue);
```

Meaning:

```
value = current state value  
setValue = function used to update the state  
initialValue = first value
```

5. Basic Counter Example

```
import { useState } from "react";  
  
export default function Counter() {  
  const [count, setCount] = useState(0);  
  
  function handleIncrease() {  
    setCount(count + 1);  
  }  
  
  return (  
    <main>  
      <h1>Counter</h1>  
      <p>Count: {count}</p>  
      <button type="button" onClick={handleIncrease}>  
        Increase  
      </button>  
    </main>  
  )  
}
```

```
    );  
  }
```

This line creates state:

```
const [count, setCount] = useState(0);
```

It means:

```
count starts at 0  
setCount updates count  
React re-renders when count changes
```

This displays state:

```
<p>Count: {count}</p>
```

This updates state:

```
setCount(count + 1);
```

6. Do Not Update State Like a Normal Variable

Wrong:

```
count = count + 1;
```

Why wrong?

Because changing a normal variable does not tell React to update the screen.

Correct:

```
setCount(count + 1);
```

Beginner rule:

To update React state, use the setter function.

7. React's Render Cycle

When state or props change, React follows this cycle:

```
State/props change
↓
Trigger update
↓
Render component functions again
↓
Compare new virtual UI with old virtual UI
↓
Commit only needed real DOM changes
```

The three main stages are:

```
Trigger
Render
Commit
```

7.1 Trigger

A trigger tells React that an update is needed.

Common triggers:

```
state changes
props change
parent component re-renders
```

Example:

```
setCount(count + 1);
```

This triggers React to update.

7.2 Render

Render means React calls your component function again.

Example:

```
function Counter() {  
  // React calls this again after state changes  
}
```

Important:

Rendering does not mean the whole browser page is rebuilt.

React re-runs the component function and creates a new description of the UI.

7.3 Commit

Commit means React updates the real browser DOM.

React compares the old UI and the new UI. Then it updates only the parts that changed.

Example:

If only the count changes from `0` to `1`, React updates only that text.

8. Virtual DOM in Simple Words

The virtual DOM is React's in-memory description of the UI.

Simple idea:

```
React keeps an old UI description.  
React creates a new UI description.  
React compares them.  
React updates only what changed in the real DOM.
```

You do not manually update the DOM like this:

```
document.querySelector("#count").textContent = count;
```

Instead, you describe the UI:

```
<p>Count: {count}</p>
```

React handles the update.

9. State Is Private to Each Component Instance

If you render the same component twice, each copy has its own state.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button type="button" onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}

export default function App() {
  return (
    <main>
      <Counter />
      <Counter />
    </main>
  );
}
```

You will see two buttons.

Clicking the first button does not change the second button.

Why?

Because each `<Counter />` has its own private `count` state.

Mental model:

First Counter has its own count.
Second Counter has its own count.
They do not share state automatically.

10. Hook Rule: Call Hooks at the Top Level

`useState` is a Hook.

Hooks must be called at the top level of a component.

Correct:

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  if (count > 5) {  
    return <p>High count</p>;  
  }  
  
  return <p>{count}</p>;  
}
```

Wrong:

```
function Counter({ enabled }) {  
  if (enabled) {  
    const [count, setCount] = useState(0);  
  }  
  
  return <p>Counter</p>;  
}
```

Do not call hooks inside:

if statements
loops

```
nested functions
event handlers
```

Correct rule:

Call hooks at the top level of your component.

Why?

React expects hooks to run in the same order every render. If hooks run sometimes but not other times, React can lose track of the state.

11. Updating State Based on Previous State

This works in simple cases:

```
setCount(count + 1);
```

But this is often safer:

```
setCount((prevCount) => prevCount + 1);
```

This is called an updater function.

Use an updater function when the next state depends on the previous state.

Example:

```
function handleIncrease() {
  setCount((prevCount) => prevCount + 1);
}
```

Read it like this:

```
React, give me the previous count.
I will return the next count.
```

Counter With Updater Function

```
import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);

  function handleIncrease() {
    setCount((prevCount) => prevCount + 1);
  }

  function handleDecrease() {
    setCount((prevCount) => prevCount - 1);
  }

  function handleReset() {
    setCount(0);
  }

  return (
    <main>
      <h1>Counter</h1>
      <p>Count: {count}</p>

      <button type="button" onClick={handleDecrease}>-</button>
      <button type="button" onClick={handleIncrease}>+</button>
      <button type="button" onClick={handleReset}>Reset</button>
    </main>
  );
}
```

Beginner rule:

If the new state depends on the old state, use the updater function.

12. Object State

State can be an object.

Example:

```
const [user, setUser] = useState({
  name: "Lina",
```

```
    age: 20,  
    city: "Colombo",  
  });
```

This is useful when several related values belong together.

Example:

```
name  
age  
city
```

These all describe one user.

13. Object State Mistake: Replacing the Whole Object

Suppose you have this state:

```
const [user, setUser] = useState({  
  name: "Lina",  
  age: 20,  
  city: "Colombo",  
});
```

Wrong update:

```
setUser({ name: "Sangeeth" });
```

This replaces the whole object.

Result:

```
{  
  name: "Sangeeth"  
}
```

You lost:

```
age  
city
```

14. Correct Object State Update With Spread

Correct:

```
setUser((prevUser) => ({  
  ...prevUser,  
  name: "Sangeeth",  
}));
```

Meaning:

```
copy the old user  
replace only name  
keep age and city
```

The spread operator copies the existing object:

```
...prevUser
```

Then this changes one field:

```
name: "Sangeeth"
```

Beginner rule:

When updating object state, copy the old object first, then change the field you want.

15. Object State With Form Inputs

A common pattern is using one state object for multiple form inputs.

```
import { useState } from "react";
```

```

export default function ProfileEditor() {
  const [user, setUser] = useState({
    name: "Lina",
    age: "20",
    city: "Colombo",
  });

  function handleChange(event) {
    const { name, value } = event.target;

    setUser((prevUser) => ({
      ...prevUser,
      [name]: value,
    }));
  }

  return (
    <main>
      <h1>Profile Editor</h1>

      <label htmlFor="name">Name</label>
      <input
        id="name"
        name="name"
        value={user.name}
        onChange={handleChange}
      />

      <label htmlFor="age">Age</label>
      <input
        id="age"
        name="age"
        value={user.age}
        onChange={handleChange}
      />

      <label htmlFor="city">City</label>
      <input
        id="city"
        name="city"
        value={user.city}
        onChange={handleChange}
      />

      <section>
        <h2>Preview</h2>
        <p>Name: {user.name}</p>
        <p>Age: {user.age}</p>
      </section>
    </main>
  );
}

```

```
        <p>City: {user.city}</p>
      </section>
    </main>
  );
}
```

16. Computed Property Names: `[name]: value`

This line is important:

```
[name]: value
```

It means:

Use the input's `name` attribute as the object property to update.

Example:

```
<input name="city" />
```

When this input changes:

```
name = "city"
```

So this code:

```
[name]: value
```

becomes like this:

```
city: value
```

This lets one `handleChange` function update many fields.

17. Array State

State can also be an array.

Example:

```
const [skills, setSkills] = useState(["JS", "CSS"]);
```

Arrays are common in React apps.

Examples:

```
skills  
products  
cart items  
todos  
comments  
notifications  
packing list items
```

In React, arrays in state should be treated as immutable.

That means:

Do not edit the old array directly. Create a new array instead.

18. Adding to Array State

Wrong:

```
skills.push("React");  
setSkills(skills);
```

This mutates the old array.

Correct:

```
setSkills((prevSkills) => [...prevSkills, "React"]);
```


Meaning:

```
copy old skills
add React at the end
return a new array
```

19. Removing From Array State

Use `filter()`.

```
setSkills((prevSkills) =>
  prevSkills.filter((skill) => skill !== "CSS")
);
```

Meaning:

```
keep every skill except CSS
return a new array
```

`filter()` does not change the old array. It creates a new array.

20. Updating One Item in Array State

Use `map()`.

Example state:

```
const [items, setItems] = useState([
  { id: "i1", text: "Shoes", packed: false },
  { id: "i2", text: "Charger", packed: false },
]);
```

Toggle one item:

```
function togglePacked(id) {
  setItems((prevItems) =>
    prevItems.map((item) =>
```

```

        item.id === id
        ? { ...item, packed: !item.packed }
        : item
    )
  );
}

```

Meaning:

```

map through all items
find the matching item
copy that item
change packed
leave other items unchanged
return a new array

```

Beginner rule:

```

spread -> add
filter -> remove
map -> update

```

21. Why Direct Mutation Is a Problem

This is a common beginner bug:

```

function List() {
  const [items, setItems] = useState(["A"]);

  function addItem() {
    items.push("B");
    setItems(items);
  }

  return <button onClick={addItem}>{items.length}</button>;
}

```

Problem:

```
items.push("B");
```

This changes the existing array.

Then:

```
setItems(items);
```

passes React the same array reference.

React may not detect the change properly because you gave it the same array object.

Correct:

```
import { useState } from "react";

export default function List() {
  const [items, setItems] = useState(["A"]);

  function addItem() {
    setItems((prevItems) => [...prevItems, "B"]);
  }

  return (
    <button type="button" onClick={addItem}>
      {items.length}
    </button>
  );
}
```

Simple rule:

Do not edit state directly. Create a new value and give that new value to React.

22. Mutation Methods to Avoid in State

Be careful with these methods when working with state:

```
push()
pop()
```

```
splice()  
sort()  
reverse()
```

They can mutate the original array.

Safer common patterns:

```
Add item: use spread  
Remove item: use filter()  
Update item: use map()  
Copy before sorting: [...items].sort()  
Copy before reversing: [...items].reverse()
```

23. Derived Values: Do Not Store What You Can Calculate

Sometimes a value can be calculated from existing state.

Example:

```
const packedCount = items.filter((item) => item.packed).length;
```

You do not need separate state for `packedCount`.

Bad idea:

```
const [packedCount, setPackedCount] = useState(0);
```

Why?

Because `packedCount` can already be calculated from `items`.

Beginner rule:

If a value can be calculated from state, do not store it as separate state.

24. Lifting State Up

Sometimes two sibling components need the same state.

Example tree:

```
App
├─ SearchInput
└─ SearchPreview
```

If both components need the search text, do not keep the state inside `SearchInput`.

Put the state in their parent.

```
import { useState } from "react";

function SearchInput({ query, onQueryChange }) {
  return (
    <input
      value={query}
      onChange={(event) => onQueryChange(event.target.value)}
      placeholder="Search..."
    />
  );
}

function SearchPreview({ query }) {
  return <p>You searched for: {query}</p>;
}

export default function App() {
  const [query, setQuery] = useState("");

  return (
    <main>
      <h1>Search</h1>
      <SearchInput query={query} onQueryChange={setQuery} />
      <SearchPreview query={query} />
    </main>
  );
}
```

This is called lifting state up.

Simple definition:

Lifting state up means moving state to the nearest parent that needs to share it.

25. `useReducer` Beginner Introduction

`useState` is your first state tool.

Do not replace every `useState` with `useReducer`.

Use `useReducer` when state logic becomes easier to describe as actions.

Examples of actions:

```
increment
decrement
reset
add item
remove item
update field
toggle item
```

Simple definition:

A reducer is a function that receives the current state and an action, then returns the next state.

26. `useReducer` Parts

`useReducer` has these main parts:

```
state = current data
dispatch = function used to send an action
action = object that describes what happened
action.type = name of the action
reducer = function that decides the next state
initialState = first state value
```

Basic pattern:

```
const [state, dispatch] = useReducer(reducer, initialState);
```

27. Counter With useReducer

```
import { useReducer } from "react";

const initialState = {
  count: 0,
};

function reducer(state, action) {
  if (action.type === "increment") {
    return { count: state.count + 1 };
  }

  if (action.type === "decrement") {
    return { count: state.count - 1 };
  }

  if (action.type === "reset") {
    return initialState;
  }

  return state;
}

export default function CounterReducer() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <main>
      <h1>Reducer Counter</h1>
      <p>Count: {state.count}</p>

      <button type="button" onClick={() => dispatch({ type: "decrement" })}>
        -
      </button>

      <button type="button" onClick={() => dispatch({ type: "increment" })}>
        +
      </button>

      <button type="button" onClick={() => dispatch({ type: "reset" })}>

```

```
        Reset
      </button>
    </main>
  );
}
```

28. How `useReducer` Works

When the user clicks this button:

```
<button type="button" onClick={() => dispatch({ type: "increment" })}>
  +
</button>
```

React sends this action:

```
{ type: "increment" }
```

The reducer receives:

```
current state
action object
```

Then this code runs:

```
if (action.type === "increment") {
  return { count: state.count + 1 };
}
```

The reducer returns the next state.

Important rule:

The reducer must return new state. It should not mutate the old state.

29. When to Use `useState` vs `useReducer`

Use `useState` when state is simple.

Examples:

```
input text
counter
show/hide menu
selected tab
single boolean
```

Use `useReducer` when state logic has several related actions.

Examples:

```
increment, decrement, reset
add item, remove item, toggle item
form with many update rules
cart with add/remove/update quantity
```

Beginner rule:

Use `useState` first. Use `useReducer` when actions make the logic easier to read.

30. Main Sprint 05 Project: Packing List

This project proves that you understand Sprint 05.

It uses:

```
useState
controlled input
form submit
preventDefault
array add with spread
array remove with filter
array update with map
stable keys
```

conditional rendering
derived value

Full Example

```
import { useState } from "react";

export default function PackingList() {
  const [newItem, setNewItem] = useState("");
  const [items, setItems] = useState([
    { id: "i1", text: "Passport", packed: false },
    { id: "i2", text: "Charger", packed: true },
  ]);

  function handleSubmit(event) {
    event.preventDefault();

    const trimmedItem = newItem.trim();

    if (trimmedItem === "") {
      return;
    }

    const item = {
      id: crypto.randomUUID(),
      text: trimmedItem,
      packed: false,
    };

    setItems((prevItems) => [...prevItems, item]);
    setNewItem("");
  }

  function handleRemove(id) {
    setItems((prevItems) => prevItems.filter((item) => item.id !== id));
  }

  function handleTogglePacked(id) {
    setItems((prevItems) =>
      prevItems.map((item) =>
        item.id === id
          ? { ...item, packed: !item.packed }
          : item
      )
    );
  }
}
```

```

const packedCount = items.filter((item) => item.packed).length;

return (
  <main>
    <h1>Packing List</h1>

    <form onSubmit={handleSubmit}>
      <label htmlFor="newItem">New item</label>
      <input
        id="newItem"
        value={newItem}
        onChange={(event) => setNewItem(event.target.value)}
      />

      <button type="submit" disabled={newItem.trim() === ""}>
        Add
      </button>
    </form>

    <p>
      Packed {packedCount} of {items.length}
    </p>

    <section>
      <h2>Items</h2>

      {items.length === 0 ? (
        <p>No items yet.</p>
      ) : (
        <ul>
          {items.map((item) => (
            <li key={item.id}>
              <label>
                <input
                  type="checkbox"
                  checked={item.packed}
                  onChange={() => handleTogglePacked(item.id)}
                />
                {item.text}
              </label>

              <button type="button" onClick={() => handleRemove(item.id)}>
                Remove
              </button>
            </li>
          ))}
        </ul>
      )}
    </section>
  </main>
)

```

```
    })  
    </section>  
  </main>  
);  
}
```

31. Packing List Explanation

State for the input

```
const [newItem, setNewItem] = useState("");
```

This stores what the user types.

State for the list

```
const [items, setItems] = useState([  
  { id: "i1", text: "Passport", packed: false },  
  { id: "i2", text: "Charger", packed: true },  
]);
```

This stores the list items.

Add item

```
setItems((prevItems) => [...prevItems, item]);
```

This creates a new array with the new item added.

Remove item

```
setItems((prevItems) => prevItems.filter((item) => item.id !== id));
```

This creates a new array without the removed item.

Toggle packed

```
setItems((prevItems) =>
  prevItems.map((item) =>
    item.id === id
      ? { ...item, packed: !item.packed }
      : item
  )
);
```

This updates one item and leaves the others unchanged.

Derived packed count

```
const packedCount = items.filter((item) => item.packed).length;
```

This calculates the number of packed items from existing state.

32. Sprint 05 Debug Task

Broken code:

```
function List() {
  const [items, setItems] = useState(["A"]);

  function addItem() {
    items.push("B");
    setItems(items);
  }

  return <button onClick={addItem}>{items.length}</button>;
}
```

Problems:

```
useState is not imported
items.push("B") mutates the old array
setItems(items) passes the same array reference
button should have type="button"
```

Fixed code:

```
import { useState } from "react";

export default function List() {
  const [items, setItems] = useState(["A"]);

  function addItem() {
    setItems((prevItems) => [...prevItems, "B"]);
  }

  return (
    <button type="button" onClick={addItem}>
      {items.length}
    </button>
  );
}
```

33. Common Sprint 05 Mistakes

Mistake 1: Changing state directly

Wrong:

```
count = count + 1;
```

Correct:

```
setCount(count + 1);
```

Mistake 2: Mutating an object

Wrong:

```
user.name = "Sangeeth";
setUser(user);
```

Correct:

```
setUser((prevUser) => ({
  ...prevUser,
  name: "Sangeeth",
}));
```

Mistake 3: Replacing the whole object by accident

Wrong:

```
setUser({ name: "Sangeeth" });
```

Correct:

```
setUser((prevUser) => ({
  ...prevUser,
  name: "Sangeeth",
}));
```

Mistake 4: Mutating an array with `push()`

Wrong:

```
items.push(newItem);
setItems(items);
```

Correct:

```
setItems((prevItems) => [...prevItems, newItem]);
```

Mistake 5: Removing with mutation

Wrong:

```
items.splice(index, 1);
setItems(items);
```

Correct:

```
setItems((prevItems) => prevItems.filter((item) => item.id !== id));
```

Mistake 6: Updating an item by editing it directly

Wrong:

```
item.packed = true;
```

Correct:

```
setItems((prevItems) =>
  prevItems.map((item) =>
    item.id === id ? { ...item, packed: true } : item
  )
);
```

Mistake 7: Calling hooks inside conditions

Wrong:

```
if (enabled) {
  const [count, setCount] = useState(0);
}
```

Correct:

```
const [count, setCount] = useState(0);
```

34. Revision Questions and Answers

Question 1: What is state?

State is data that React remembers inside a component. It can change over time and control what appears on the screen.

Question 2: What does `useState` return?

It returns an array with two values:

```
current state value  
setter function
```

Example:

```
const [count, setCount] = useState(0);
```

Question 3: Why should you use the setter function?

Because the setter tells React that state changed and the UI may need to re-render.

Question 4: What are trigger, render, and commit?

```
Trigger = React is told an update is needed  
Render = React calls component functions again  
Commit = React updates the real DOM
```

Question 5: Why is direct mutation a problem?

Direct mutation changes the old object or array. React may not detect the change properly because the reference can stay the same.

Question 6: How do you update object state safely?

Copy the old object with spread and change only the needed field.

```
setUser((prevUser) => ({  
  ...prevUser,  
  name: "Sangeeth",  
}));
```

Question 7: What does `[name]: value` do?

It uses the input's `name` attribute as the object property to update.

Question 8: How do you add an item to array state?

Use spread:

```
setItems((prevItems) => [...prevItems, newItem]);
```

Question 9: How do you remove an item from array state?

Use `filter()`:

```
setItems((prevItems) => prevItems.filter((item) => item.id !== id));
```

Question 10: How do you update one item in array state?

Use `map()`:

```
setItems((prevItems) =>
  prevItems.map((item) =>
    item.id === id ? { ...item, packed: !item.packed } : item
  )
);
```

Question 11: What is lifting state up?

Lifting state up means moving state to the nearest parent component that needs to share it with children.

Question 12: When is `useReducer` useful?

`useReducer` is useful when state logic has several related actions and is easier to read as action-based logic.

35. Sprint 05 Mini Revision Sheet

State

React's memory for data that changes.

```
const [count, setCount] = useState(0);
```

Setter Function

Used to update state.

```
setCount(count + 1);
```

Updater Function

Used when next state depends on previous state.

```
setCount((prevCount) => prevCount + 1);
```

Render Cycle

```
Trigger -> Render -> Commit
```

Object Update

```
setUser((prevUser) => ({  
  ...prevUser,  
  name: "New name",  
}));
```

Object Form Update

```
const { name, value } = event.target;  
  
setUser((prevUser) => ({  
  ...prevUser,  
  [name]: value,  
}));
```

Array Add

```
setItems((prevItems) => [...prevItems, newItem]);
```

Array Remove

```
setItems((prevItems) => prevItems.filter((item) => item.id !== id));
```

Array Update

```
setItems((prevItems) =>
  prevItems.map((item) =>
    item.id === id ? { ...item, done: !item.done } : item
  )
);
```

Lifting State Up

Move state to the nearest shared parent.

useReducer

Use it when state logic is clearer as actions.

```
const [state, dispatch] = useReducer(reducer, initialState);
```

36. Definition of Done for Sprint 05

Sprint 05 is complete when you can:

- explain what state is,
- create state with `useState`,
- update state with a setter function,
- explain trigger, render, and commit,
- explain why each component instance has private state,
- call hooks only at the top level,
- use updater functions when needed,
- update object state with spread,

- use `[name]: value` for object form updates,
 - add to array state with spread,
 - remove from array state with `filter()`,
 - update one array item with `map()`,
 - explain why direct mutation is risky,
 - avoid `push()`, `pop()`, and `splice()` on state arrays,
 - calculate derived values instead of storing unnecessary state,
 - lift state up when siblings need shared data,
 - explain basic `useReducer`,
 - build a reducer counter,
 - build the Packing List app.
-

37. Suggested Study Order

Study Sprint 05 in this order:

1. What state is
2. Basic `useState`
3. Counter with `useState`
4. React render cycle
5. Trigger, render, commit
6. Private state per component instance
7. Hook top-level rule
8. Updating from previous state
9. Object state
10. Object updates with spread
11. `[name]: value`
12. Array state
13. Add with spread
14. Remove with `filter()`
15. Update with `map()`
16. Why mutation is a problem
17. Derived values
18. Lifting state up
19. Beginner `useReducer`
20. Packing List project
21. Debug task
22. Self-check questions

Do not only memorize the rules.

Build the examples. Break them. Fix them.

That is how Sprint 05 becomes clear.

38. Final Beginner Explanation

Sprint 05 teaches this:

I can use state to let React remember changing data. I use `useState` to create state and a setter function to update it. When state changes, React triggers an update, renders the component again, compares the new UI with the old UI, and commits only the needed DOM changes. I should not mutate objects or arrays in state. Instead, I create new objects and arrays using spread, `filter()`, and `map()`. I can lift state up when multiple components need the same data. I can also use `useReducer` when state logic is easier to describe with actions like increment, decrement, reset, add, remove, or toggle.

39. Shortest Correct Mental Model

State is React's memory.
When state changes, React re-renders.
Do not edit objects or arrays directly.
Create new objects and arrays.
Use `useState` first.
Use `useReducer` when action-based logic is clearer.

40. Practice Prompt

After finishing Sprint 05, explain this out loud:

State is data React remembers. I create state with `useState`. I update it with the setter function. When state changes, React re-renders and updates the screen. I should not mutate state directly. For objects, I copy with spread. For arrays, I add with spread, remove with `filter`, and update with `map`. If sibling components need the same state, I lift the state to their parent. If state logic has many action types, I can use `useReducer`.

If you can explain that clearly and build the Packing List project, Sprint 05 is complete.